

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a32c0660e58957c17.jsonl`  
- SHA-256: `d3aa37879875afb9b7cb7ec509df294872a1f6edfaa35ffbd016ae14332199`  
- Source modified: `2026-06-10T16:24:35+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:22:08.041Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: Key rotation overwrites the only copy of the current key before the file is rekeyed; rollback misses realistic failures
SEVERITY CLAIMED: critical
FILE: src/muneem/cli.py (line 438-446)
DESCRIPTION: `db rotate-key` writes the fresh key into the keychain FIRST (`db_key.set_key(fresh)`), destroying the only durable copy of the current key (it survives only in process memory as `current`), and only then rekeys the file. Two realistic failure modes permanently lock the ledger: (1) a crash/power loss/Ctrl-C between `set_key(fresh)` and a completed `PRAGMA rekey` leaves keychain=fresh, file=old-key, old key gone; (2) the rollback `except` only catches `dbmod.EncryptedDatabaseError`, but `rekey_database` (src/muneem/db.py:274-279) can raise `sqlcipher3.OperationalError` (e.g. 'database is locked' ? very plausible on Windows with another process/AV holding the file) or any other sqlcipher3 error from `PRAGMA rekey`, which escapes uncaught, skips the rollback, and exits with keychain=fresh / file=old. Unless the user happens to have a recovery file (which wraps the OLD key), the entire financial ledger and all same-key backups are unrecoverable. The inline comment ('Keychain-first with rollback: if the file rekey fails, the keychain still matches the file') asserts the opposite of what the code does ? keychain-first guarantees a mismatch window.
EVIDENCE QUOTED: # Keychain-first with rollback: if the file rekey fails, the keychain still matches the file.`

```
db_key.set_key(fresh)  
try:  
    dbmod.rekey_database(db_path, current, fresh)  
except dbmod.EncryptedDatabaseError as exc:  
    db_key.set_key(current) # roll back; the file is still under the old key
```

SUGGESTED FIX: Never have a moment where only one key exists in durable storage: write the fresh key under a secondary keychain name (e.g. 'db-key-pending') first, rekey the file, then promote it to 'db-key' and delete the pending entry; on open failure, fall back to trying the pending key. At minimum: rekey the file BEFORE touching the 'db-key' entry, broaden the except to catch all exceptions (re-raising after rollback), and stash the old key under 'db-key-prev' before overwriting.

Your job is to try to REFUTE it. Your lens: reachability and impact: check whether the scenario can actually occur in realistic single-user use on Windows, and whether the assigned severity is justified.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file

existence and git-tracking status (git ls-files, git log --all -- <path>).

- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Structured output only.

2. user (2026-06-10T16:22:13.471Z)

```
380         else:
381             state = "encrypted (SQLCipher)"
382             typer.echo(f"ledger : {db_path}")
383             typer.echo(f"state  : {state}")
384             typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
390 dir)."),
391     ) -> None:
392         """Migrate an existing *plaintext* ledger to an encrypted one (SQLCipher), in
393 place."""
394         from muneem import db as dbmod
395         from muneem.db_key import get_or_create_key
396
397         db_path = db or load_config().db_path
398         if not Path(db_path).exists():
399             typer.echo(
400                 f"nothing to encrypt: {db_path} does not exist (a new ledger is born
401 encrypted)."
402             )
403             return
404         if not dbmod.is_plaintext_sqlite(db_path):
405             typer.echo(
406                 f"{Path(db_path).name} is already encrypted (or not a SQLite file); nothing
407 to do."
408             )
409             return
410         n = dbmod.encrypt_database(db_path, get_or_create_key())
411         typer.echo(
412             f"encrypted {Path(db_path).name}: {n} table(s) migrated. A plaintext backup
413 remains at "
414             f"{Path(db_path).name}.plaintext.bak ? verify, then securely delete it."
415         )
416
417
418     def _require_encrypted_ledger(db_path: Path) -> str:
419         """Return the ledger's keychain key, or exit with a clear message if it can't be
420 used."""
421         from muneem import db as dbmod
422         from muneem.db_key import peek_key
423
424         if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
425             typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).",
426 err=True)
427             raise typer.Exit(1)
428         key = peek_key()
429         if not key:
430             typer.echo("no encryption key in the keychain for this ledger.", err=True)
431             raise typer.Exit(1)
432         return key
```

```

427
428     @db_app.command("rotate-key")
429     def db_rotate_key(
430         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
431 dir)."),
432     ) -> None:
433         """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
434         from muneem import db as dbmod
435         from muneem import db_key
436
437         db_path = db or load_config().db_path
438         current = _require_encrypted_ledger(db_path)
439         fresh = db_key.new_key()
440         # Keychain-first with rollback: if the file rekey fails, the keychain still matches
441         the file.
442         db_key.set_key(fresh)
443         try:
444             dbmod.rekey_database(db_path, current, fresh)
445         except dbmod.EncryptedDatabaseError as exc:
446             db_key.set_key(current) # roll back; the file is still under the old key
447             typer.echo(str(exc), err=True)
448             raise typer.Exit(1) from exc
449             typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
450
451     @db_app.command("backup")
452     def db_backup(
453         dest: Path = typer.Argument(..., help="Destination path for the encrypted backup
454 copy."),
455         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
456 dir)."),
457     ) -> None:
458         """Write an encrypted online-backup copy of the ledger (consistent even with WAL
459 active)."""
460         from muneem import db as dbmod
461
462         db_path = db or load_config().db_path
463         key = _require_encrypted_ledger(db_path)
464         try:
465             dbmod.backup_database(db_path, dest, key)
466         except dbmod.EncryptedDatabaseError as exc:
467             typer.echo(str(exc), err=True)
468             raise typer.Exit(1) from exc
469             typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")
470
471     @db_app.command("setup-recovery")
472     def db_setup_recovery(
473         out: Path | None = typer.Option(
474             None, help="Recovery file path (default: <data dir>/muneem-recovery.json)."
475         ),
476     ) -> None:
477         """Create a passphrase-wrapped recovery file for the ledger's encryption key.
478
479         Lets you restore the key (and thus the ledger + backups) if the OS keychain is ever
480         lost. The
481         file holds only the *wrapped* key ? useless without the passphrase.
482         """
483         import getpass
484
485         from muneem import recovery
486         from muneem.db_key import peek_key
487
488         key = peek_key()
489         if not key:

```

```

486         typer.echo("no encryption key in the keychain yet (create the ledger first).",
err=True)
487         raise typer.Exit(1)
488     out_path = out or (load_config().data_dir / "muneem-recovery.json")
489     passphrase = getpass.getpass("New recovery passphrase: ")

```

3. user (2026-06-10T16:22:15.459Z)

```

11-import os
12-import sqlite3
13-from collections.abc import Iterator
14-from contextlib import contextmanager
15-from pathlib import Path
16-
17:from muneem.errors import EncryptedDatabaseError # re-exported so db.EncryptedDatabaseError
works
18-
19-SCHEMA_VERSION = 4
20-
21-# Unencrypted SQLite files start with this 16-byte magic header; an encrypted (SQLCipher)
22-# file does not, so we can tell them apart without a key.
23-__SQLITE_MAGIC = b"SQLite format 3\x00"
24-
25-
26-
27-
28-
29-
30-
31-
32-
33-
34-
35-
36-
37-
38-
39-
40-
41-
42-
43-
44-
45-
46-
47-
48-
49-
50-
51-
52-
53-
54-
55-
56-
57-
58-
59-
60-
61-
62-
63-
64-
65-
66-
67-
68-
69-
70-
71-
72-
73-
74-
75-
76-
77-
78-
79-
80-
81-
82-
83-
84-
85-
86-
87-
88-
89-
90-
91-
92-
93-
94-
95-
96-
97-
98-
99-
100-
101-
102-
103-
104-
105-
106-
107-
108-
109-
110-
111-
112-
113-
114-
115-
116-
117-
118-
119-
120-
121-
122-
123-
124-
125-
126-
127-
128-
129-
130-
131-
132-
133-
134-
135-
136-
137-
138-
139-
140-
141-
142-
143-
144-
145-
146-
147-
148-
149-
150-
151-
152-
153-
154-
155-
156-
157-
158-
159-
160-
161-
162-
163-
164-
165-
166-
167-
168-
169-
170-
171-
172-
173-
174-
175-
176-
177-
178-
179-
180-
181-
182-
183-
184-
185-
186-
187-
188-
189-
190-
191-
192-
193-
194-
195-
196-
197-
198-
199-
200-
201-
202-
203-
204-
205-
206-
207-
208-
209-
210-
211-
212-
213-

```

```

214- The original is preserved as ``<name>.plaintext.bak`` for the caller to verify and then
215- securely delete (it still holds the cleartext data). Returns the number of user tables
216- copied. Raises :class:`EncryptedDatabaseError` if ``db_path`` is not a plaintext SQLite
file.
217- """
218- import sqlcipher3
219-
220- path = Path(db_path)
221- if not is_plaintext_sqlite(path):
222-     raise EncryptedDatabaseError(
223-         f"{path.name} is not a plaintext SQLite database (already encrypted or missing)"
224-     )
225-
226- enc_tmp = path.with_name(path.name + ".enc.tmp")
227- enc_tmp.unlink(missing_ok=True)
228- try:
229-     --
244-     with connect(str(enc_tmp), key=key) as check:
245-         integrity = check.execute("PRAGMA integrity_check").fetchone()[0]
246-         ntables = check.execute(
247-             "SELECT count(*) FROM sqlite_master WHERE type = 'table'"
248-         ).fetchone()[0]
249-         if integrity != "ok" or ntables != src_tables:
250-             raise EncryptedDatabaseError(
251-                 f"encryption migration verification failed for {path.name} "
252-                 f"(integrity={integrity!r}, tables {ntables}/{src_tables}); original left
untouched"
253-             )
254-
255-         backup = path.with_name(path.name + ".plaintext.bak")
256-         backup.unlink(missing_ok=True)
257-     --
260- finally:
261-     # Never leave a partial temp behind: no-op on success (already renamed), cleans up
on error.
262-     enc_tmp.unlink(missing_ok=True)
263-     return int(ntables)
264-
265-
266-def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267-     """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268-
269-     Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270-     :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271-     muneem ledger. The caller is responsible for updating the keychain entry to ``new_key``
only
272-     after this returns successfully.
273-     """
274-     conn = _open_encrypted(str(db_path), current_key) # validates current_key
275-     try:
276-         conn.execute(f"PRAGMA rekey = '{new_key}'")
277-     --
282-
283-def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284-     """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285-
286-     Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL active,
so no
287-     naive file copy and no manual checkpoint needed. The destination is keyed with the same
key, so
288-     it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is wrong.
289-     """
290-     import sqlcipher3
291-

```

```

292-     dest_path = Path(dest)
293-     dest_path.parent.mkdir(parents=True, exist_ok=True)
294-     dest_path.unlink(missing_ok=True)

```

4. user (2026-06-10T16:22:22.574Z)

```

264
265
266     def rekey_database(db_path: Path | str, current_key: str, new_key: str) -> None:
267         """Re-encrypt the ledger under ``new_key`` in place (SQLCipher ``PRAGMA rekey``).
268
269         Opens with ``current_key`` first (which verifies it), then rekeys. Raises
270         :class:`EncryptedDatabaseError` if ``current_key`` is wrong or the file is not an
encrypted
271         muneem ledger. The caller is responsible for updating the keychain entry to
``new_key`` only
272         after this returns successfully.
273         """
274         conn = _open_encrypted(str(db_path), current_key) # validates current_key
275         try:
276             conn.execute(f"PRAGMA rekey = \x'{new_key}'")
277             conn.commit()
278         finally:
279             conn.close()
280         _harden_permissions(str(db_path))
281
282
283     def backup_database(db_path: Path | str, dest: Path | str, key: str) -> None:
284         """Write an **encrypted** online-backup copy of the ledger to ``dest`` (same key).
285
286         Uses the SQLite Online Backup API for a consistent snapshot ? safe even with WAL
active, so no
287         naive file copy and no manual checkpoint needed. The destination is keyed with the
same key, so
288         it stays encrypted at rest. Raises :class:`EncryptedDatabaseError` if ``key`` is
wrong.
289         """
290         import sqlcipher3
291
292         dest_path = Path(dest)
293         dest_path.parent.mkdir(parents=True, exist_ok=True)
294         dest_path.unlink(missing_ok=True)
295         source = _open_encrypted(str(db_path), key) # validates key
296         try:
297             target = sqlcipher3.connect(str(dest_path))
298             try:
299                 target.execute(_raw_key_pragma(key)) # key the destination so the backup is
encrypted
300                 source.backup(target) # consistent snapshot of the live DB
301             finally:
302                 target.close()
303         finally:

```

5. user (2026-06-10T16:22:22.810Z)

```

1     """The ledger's encryption key ? minted once, kept in the OS keychain.
2
3     muneem's local ledger is **encrypted at rest** with SQLCipher (DESIGN.md §5.4 rule 4).
4     The 256-bit key is generated on first use and stored in the OS keychain via
5     :mod:`muneem.secrets` (Windows Credential Manager / macOS Keychain / Secret Service) ?
6     never on disk in plaintext, never in the repo, never logged.
7
8     It is handed to SQLCipher in **raw-key form** (``PRAGMA key = "\x'<64 hex>'``), which
uses
9     the 256-bit key directly and *skips* the passphrase KDF. That is correct here because

```

```

the
10     key is already full-entropy random; KDF stretching only buys security for low-entropy
human
11     passphrases (an opt-in we may add later ? see ROADMAP D.2 phased plan).
12     """
13
14     from __future__ import annotations
15
16     import secrets as _stdlib_secrets
17
18     from muneem import secrets as _keychain
19
20     # Keychain entry name (under the shared "muneem" service) holding the 64-hex-char DB
key.
21     _DB_KEY_NAME = "db-key"
22
23
24     def get_or_create_key() -> str:
25         """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first
use."""
26         existing = _keychain.get_secret(_DB_KEY_NAME)
27         if existing:
28             return existing
29         key = _stdlib_secrets.token_bytes(32).hex()
30         _keychain.set_secret(_DB_KEY_NAME, key)
31         return key
32
33
34     def peek_key() -> str | None:
35         """Return the stored key, or ``None`` if none has been minted yet. Never creates
one."""
36         return _keychain.get_secret(_DB_KEY_NAME)
37
38
39     def new_key() -> str:
40         """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
41         return _stdlib_secrets.token_bytes(32).hex()
42
43
44     def set_key(key: str) -> None:
45         """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
46         _keychain.set_secret(_DB_KEY_NAME, key)
47

```

6. user (2026-06-10T16:22:48.037Z)

```

C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-27-      conn.execute("CREATE TABLE
t (id INTEGER PRIMARY KEY, amount REAL)")
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-28-      conn.executemany("INSERT
INTO t (amount) VALUES (?)", [(float(i),) for i in range(n)])
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-29-
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-30-
tests\test_db_crypto_ops.py:31:# --- rekey / rotation
-----
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-32-
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-33-
tests\test_db_crypto_ops.py:34:def test_rekey_changes_the_key_and_keeps_data(tmp_path):
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-35-     p = tmp_path / "led.db"
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-36-     _seed_encrypted(p, KEY_A, n=5)
tests\test_db_crypto_ops.py:37:     dbmod.rekey_database(p, KEY_A, KEY_B)
tests\test_db_crypto_ops.py-38-     with pytest.raises(dbmod.EncryptedDatabaseError): # old key
no longer opens it
tests\test_db_crypto_ops.py-39-         with dbmod.connect(p, key=KEY_A):
C:\Users\avidu\Projects\muneem\tests\test_db_crypto_ops.py-40-             pass

```

```

tests\test_db_crypto_ops.py-41-     with dbmod.connect(p, key=KEY_B) as conn: # new key works,
data intact
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-42-         assert
conn.execute("SELECT count(*) FROM t").fetchone()[0] == 5
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-43-
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-44-
tests\test_db_crypto_ops.py:45:def test_rekey_with_wrong_current_key_raises(tmp_path):
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-46-     p = tmp_path / "led.db"
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-47-     _seed_encrypted(p, KEY_A)
tests\test_db_crypto_ops.py-48-     with pytest.raises(dbmod.EncryptedDatabaseError):
tests\test_db_crypto_ops.py:49:         dbmod.rekey_database(p, KEY_B, KEY_A)
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-50-
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-51-
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-52-# --- encrypted backup
-----
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-53-
--
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-114-         pass
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-115-     assert "muneeem db encrypt" in
str(ei.value) # actionable guidance, not a cryptic error
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-116-
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-117-
tests\test_db_crypto_ops.py:118:# --- CLI: db rotate-key / db backup
-----
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-119-
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-120-
tests\test_db_crypto_ops.py-121-def _new_encrypted_ledger(path):
tests\test_db_crypto_ops.py-122-     with dbmod.connect_encrypted(path) as conn:
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-123-         dbmod.init_db(conn)
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-124-
dbmod.insert_document(conn, source="file", sha256="s", status="parsed")
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-125-
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-126-
tests\test_db_crypto_ops.py:127:def test_db_rotate_key_cli(tmp_path):
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-128-     led = tmp_path / "ledger.db"
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-129-     _new_encrypted_ledger(led)
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-130-     old = db_key.peek_key()
tests\test_db_crypto_ops.py:131:     res = runner.invoke(app, ["db", "rotate-key", "--db",
str(led)])
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-132-     assert res.exit_code == 0,
res.output
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-133-     new = db_key.peek_key()
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-134-     assert new and new != old
tests\test_db_crypto_ops.py-135-     with dbmod.connect(led, key=new) as conn: # opens under the
new key
--
tests\test_db_crypto_ops.py-149-     with dbmod.connect(dest, key=db_key.peek_key()) as conn:
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-150-         assert
dbmod.get_document_by_sha(conn, "s") is not None
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-151-
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-152-
tests\test_db_crypto_ops.py:153:def
test_db_rotate_key_refuses_when_no_encrypted_ledger(tmp_path):
tests\test_db_crypto_ops.py:154:     res = runner.invoke(app, ["db", "rotate-key", "--db",
str(tmp_path / "nope.db")])
C:\Users\avidu\Projects\muneeem\tests\test_db_crypto_ops.py-155-     assert res.exit_code == 1 and
"db encrypt" in res.output

```

7. user (2026-06-10T16:22:48.869Z)

- 1 """"Secret storage via the OS keychain.
- 2
- 3 OAuth tokens, cached statement passwords, and the DB encryption key are stored here (Windows
- 4 Credential Manager / macOS Keychain / Secret Service), never in the repo, on disk in


```

plaintext, or
5         in logs. See DESIGN.md section 5.
6
7         External-call discipline (every keyring call is wrapped with attribution):
8         - **read** (:func:`get_secret`) is *graceful* ? a missing or unavailable backend returns
`None`
9         (absence is a valid state; this also keeps headless CI happy).
10        - **write** (:func:`set_secret`) is *loud* ? a failure raises
:class:`muneem.errors.KeychainError`,
11        because silently losing a write (e.g. the DB key) would be dangerous. The secret
*name* is
12        included for diagnosis; the value never is.
13        ""
14
15        from __future__ import annotations
16
17        import keyring
18        from keyring.errors import KeyringError, PasswordDeleteError
19
20        from muneem.errors import KeychainError
21
22        _SERVICE = "muneem"
23
24
25        def get_secret(name: str) -> str | None:
26            """Return the stored secret, or `None` if absent or the keychain is
unavailable."""
27            try:
28                return keyring.get_password(_SERVICE, name)
29            except KeyringError:
30                return None
31
32
33        def set_secret(name: str, value: str) -> None:
34            """Store (or replace) a secret in the OS keychain. Raises KeychainError on
failure."""
35            try:
36                keyring.set_password(_SERVICE, name, value)
37            except KeyringError as exc:
38                # Name only ? never the value ? in the message (it may be a password or the DB
key).
39                raise KeychainError(f"could not store secret {name!r} in the OS keychain") from
exc
40
41
42        def delete_secret(name: str) -> None:
43            """Delete a secret if present; a no-op if it is not (or the keychain is
unavailable)."""
44            try:
45                keyring.delete_password(_SERVICE, name)
46            except (PasswordDeleteError, KeyringError):
47                pass
48

```

8. user (2026-06-10T16:23:36.646Z)

```

1        """muneem's exception hierarchy.
2
3        Every error muneem raises derives from :class:`MuneemError`, so callers can catch the
whole family
4        with one `except`. Two rules for messages on these exceptions, because they may end up
in logs:
5
6        1. **Only non-sensitive context.** A file *name* is acceptable (it's the user's own
machine), but

```

```

7         never raw statement text, passwords, balances, or account numbers. Use
8         :mod:`muneem.redaction` helpers to sanitise anything derived from a document.
9     2. **Chain the cause.** Raise with ``raise ParseError(...) from exc`` so the original
traceback is
10         available for local debugging while the muneem-level message stays clean.
11     """
12
13     from __future__ import annotations
14
15
16     class MuneemError(Exception):
17         """Base class for all errors raised by muneem."""
18
19
20     class ParseError(MuneemError):
21         """A document could not be parsed (corrupt PDF, unexpected layout, extraction
failure).
22
23         Carries only non-sensitive context (e.g. the file name and the failing stage) ?
never the
24         statement contents or the underlying library's message verbatim.
25         """
26
27
28     class UnlockError(MuneemError):
29         """A password-protected document could not be unlocked (no working password, bad
input)."""
30
31
32     class EncryptedDatabaseError(MuneemError):
33         """The encrypted ledger could not be opened ? wrong key, missing SQLCipher binding,
or a
34         plaintext file where an encrypted one was expected."""
35
36
37     class KeychainError(MuneemError):
38         """Storing a secret in the OS keychain failed (no backend, locked keychain, etc.).
39
40         The secret *name* is safe to include; never the value."""
41
42
43     class ConfigError(MuneemError):
44         """A user config file (e.g. password_rules.yaml) is malformed.
45
46         Messages must reference the file by name only ? never echo its contents, which may
hold
47         password material."""
48
49
50     class MigrationError(MuneemError):
51         """A schema migration could not be applied, or the on-disk schema diverged from the
52         migration history (e.g. the frozen baseline was edited without adding a
migration)."""
53

```

9. user (2026-06-10T16:23:37.830Z)

docs\CURRENT_STATE.md:42: * CLI: `ingest <pdf>` (promo, or `--doc-type bank\_statement` ? ****auto-unlocks encrypted PDFs****, see [ingesting-statements.md](./ingesting-statements.md)), `offers list`, `txns list <bank>`, `eval <folder>` (reconcile-rate report ? the seed?golden onboarding loop, see [onboarding-a-bank.md](./onboarding-a-bank.md)), and `db` encryption ops (`status`, `encrypt`, `rotate-key`, `backup`, `setup-recovery`, `recover` ? see [encryption-at-rest.md](./encryption-at-rest.md)).

docs\CURRENT_STATE.md:55: * ****Recover the May 2025 vintage****: the clusterer drops one transaction line ? incomplete ledger; needs smarter line recovery. Isolated and safe (it currently stores

nothing; June 2025 was recovered via balance-delta reconstruction).

docs\DEPENDENCIES.md:41:| cryptography | Apache-2.0 / BSD-3 | Argon2id + AES-GCM for the passphrase-wrapped DB-key recovery file (PyCA) |

docs\encryption-at-rest.md:42:### Disaster recovery ? the key-loss problem (solved)

docs\encryption-at-rest.md:46:*and every backup*. So a **passphrase-wrapped recovery file** is implemented (``muneem.recovery``):

docs\encryption-at-rest.md:48:- **``muneem db setup-recovery``** ? wraps the current DB key under a user passphrase

docs\encryption-at-rest.md:49: (**``Argon2id``** KDF ? **``AES-256-GCM``**) into ``muneem-recovery.json``. The file holds only the **wrapped**

docs\encryption-at-rest.md:51: **off-machine** (password manager / separate drive). The pre-commit guard blocks ``recovery.json``

docs\encryption-at-rest.md:59:distinct from the recovery file above, which keeps the frictionless keychain key as primary.

docs\feedbackJune2026.md:37:| "muneem db backup (proper Online Backup API, not naive cp)" | **Done in #39.** ``backup_database()`` uses the SQLite Online Backup API (WAL-safe), destination keyed ? encrypted backup. | A documented restore runbook; optional **recovery-key** backups (see §2). |

docs\feedbackJune2026.md:38:| "key rotation scaffolding (the deferred passphrase path)" | **Rotation done in #39.** ``rekey_database()`` + ``muneem db rotate-key`` (keychain-first with rollback). | The **passphrase path** specifically ? but reframed as **recovery**, not just "stronger" (see §2). |

docs\feedbackJune2026.md:47:## 2. ? The biggest risk the review **under-weights**: key-loss / disaster recovery (P0)

docs\feedbackJune2026.md:59:**The right way** (reframe the deferred "passphrase mode" from security ? recovery):

docs\feedbackJune2026.md:61:- Add a **recovery wrapper**: encrypt the DB key under a user passphrase (Argon2id, OWASP params)

docs\feedbackJune2026.md:62: and store that wrapped blob in a ``recovery.json`` the user backs up **off-machine** (password

docs\feedbackJune2026.md:63: manager / printed). Losing the keychain ? restore the key from passphrase + recovery file.

docs\feedbackJune2026.md:66:- Optionally let ``db backup --recovery-key`` write a backup under the passphrase-derived key, so a

docs\feedbackJune2026.md:68:- Ship a **docs/recovery.md** runbook: "keychain lost," "move to a new machine," "verify a backup."

docs\feedbackJune2026.md:70:**DECISION**: which recovery model ? (a) passphrase-wrapped recovery file **(recommended)**, (b)

docs\feedbackJune2026.md:154:### P9 ? Observability / ops surface . ? Accept (partly done; pull recovery forward)

docs\feedbackJune2026.md:159:(date/amount/account/status), light analytics. **Crucially**, the recovery model (§2) belongs here but

docs\feedbackJune2026.md:161:(recovery) + Phase H (the rest).

docs\feedbackJune2026.md:183:- **Phase A ? Foundation & recovery (low-risk, high-leverage)**: **recovery model (P0)** .

docs\feedbackJune2026.md:204:1. **Recovery / key-loss model** ? **passphrase-wrapped recovery file** (Argon2id + AES-GCM). ? shipped (#43).

docs\feedbackJune2026.md:231:- **And the addition the review missed**: key-loss recovery is P0 ? above most of the feature list.

docs\feedbackJune2026.md:263:**Next step**: §5's settled decisions are being implemented (recovery #43, migrations #44, Windows CI

docs\ROADMAP.md:36:> **Remaining**: recover the dropped line (smarter line recovery in the clusterer) for May 2025 and

docs\ROADMAP.md:48:2. **Axis May 2025 recovery** ? recover the one transaction line the clusterer drops (incomplete

docs\ROADMAP.md:95:| Table recovery (born-digital statements) | **pdfplumber** | Coordinate/word-level extraction (HDFC clustering; Axis/AU via the shared savings path). |

docs\ROADMAP.md:154:- [x] **A.4 (M)** ``muneem ingest <pdf>`` writes a ``documents`` + ``offers`` row; ``muneem offers list`` reads them back. **Ground-truth + regression test**: establish the known promo codes for each of the 10 corpus PDFs, then a ``pytest`` that runs the pipeline over ``testdata/promo-emails/`` and asserts recovery. (Building the ground-truth answer key is itself part of this task.)

[Showing results with pagination = limit: 30]

10. user (2026-06-10T16:24:31.853Z)

Structured output provided successfully

11. assistant (2026-06-10T16:24:34.943Z)

Verification complete.